

Project 6

Automated Lead Ingestion Engine

Event-Driven Workflow Architecture Using n8n, API Integrations, and CRM Synchronization

I carefully reviewed your project assessment document before writing this. Below is the full in-depth Google Docs-ready documentation.

1. Introduction and Motivation

Modern B2B sales pipelines depend heavily on automated lead generation systems. Businesses often collect leads from LinkedIn scraping tools, enrichment platforms, and external data providers. However, integrating these systems into a CRM while handling rate limits, deduplication, asynchronous execution, and error handling is complex.

This project focuses on building a production-grade, event-driven lead ingestion architecture using:

- n8n workflow automation
- PhantomBuster for scraping
- Apollo.io for enrichment
- HubSpot CRM for storage

Rather than building a simple linear automation, the architecture was designed to reflect real-world production systems with clear separation of responsibilities, asynchronous execution, error isolation, and API safety mechanisms.

The primary goal was to design a scalable, fault-tolerant, API-compliant automation pipeline suitable for real enterprise deployment.

2. Problem Statement

Lead ingestion systems must address multiple challenges:

1. Scraping tasks may take several minutes.
2. CRM APIs have strict rate limits.
3. Duplicate contacts must be prevented.
4. Enrichment APIs may fail or hit quota limits.
5. Processing must not block downstream execution.
6. Failures in scraping should not break ingestion logic.

The system must:

- Launch scraping jobs
 - Detect completion asynchronously
 - Retrieve data safely
 - Enrich leads
 - Deduplicate contacts
 - Insert or update CRM records
 - Remain resilient under API constraints
-

3. High-Level System Architecture

The system was intentionally divided into two independent workflows:

Workflow 1 → Scraping Job Launcher

Workflow 2 → Lead Processing Pipeline

This separation was critical for:

- Scalability
- Error isolation
- Asynchronous execution
- Clear separation of concerns

The architecture follows an event-driven design rather than polling.

4. Workflow 1 – Scraping Job Launcher

4.1 Purpose

This workflow is responsible only for triggering PhantomBuster scraping jobs. It does not process data.

4.2 Execution Flow

1. Scheduled trigger (daily or weekly).
2. API call to PhantomBuster Launch Agent endpoint.
3. Agent ID and search parameters passed dynamically.
4. Webhook URL for completion notification provided.

After the job is launched, the workflow ends immediately.

4.3 Design Rationale

This design prevents:

- Long blocking executions
- Resource wastage
- Tight coupling between scraping and processing
- Polling inefficiencies

It mirrors microservice-like decoupling.

5. Workflow 2 – Lead Processing Pipeline

This workflow begins only after PhantomBuster sends a completion webhook.

5.1 Trigger Mechanism

Webhook-based event-driven execution ensures:

- Immediate processing
 - No manual polling
 - Reduced API overhead
 - Precise execution timing
-

5.2 Lead Fetching

After receiving the webhook:

- Scraped leads are retrieved via PhantomBuster API

- Data is validated
 - Raw JSON payload parsed
-

5.3 Batch Processing Strategy

Leads are processed in batches of 5–10 records.

This was intentionally designed to:

- Avoid HubSpot rate limits
- Prevent HTTP 429 errors
- Maintain API safety compliance

A delay node is inserted between batches to buffer requests.

5.4 Apollo.io Enrichment

Each lead is enriched using Apollo API:

- Email validation
- Work email retrieval
- Additional company metadata

If enrichment fails:

- Continue-on-fail logic enabled
- Partial data is preserved
- Workflow continues execution

This prevents pipeline collapse due to enrichment quota limits.

5.5 Deduplication Strategy

HubSpot “Create or Update” (Upsert) operation is used.

HubSpot automatically deduplicates based on email address.

Benefits:

- Prevents duplicate records
- Maintains data consistency

- Simplifies logic
 - Uses native CRM behavior
-

6. Error Handling and Fault Tolerance

The system includes:

- Continue-on-fail API nodes
- Structured logging
- Batch delay buffering
- Error isolation between workflows

Scraping failure does not affect lead processing logic.

Enrichment failure does not stop CRM insertion.

This creates a fault-tolerant automation system.

7. API Safety and Rate Limit Compliance

HubSpot Free Tier API limits are strict.

To handle 200+ contacts safely:

- Small batch sizes
- Delay nodes between calls
- Request buffering
- Controlled throughput

This prevents:

- HTTP 429 errors
 - Account suspension
 - Workflow crashes
-

8. Asynchronous Event-Driven Design

Instead of polling for job completion, webhook-based architecture was implemented.

Advantages:

- No constant API requests
- Reduced server load
- Faster execution
- Clean execution boundary
- Scalable to higher workloads

This design reflects modern distributed system principles.

9. Scalability Considerations

Because workflows are independent:

- Scraping frequency can increase independently.
- Enrichment logic can be extended.
- CRM target can be replaced.
- Batch size can be tuned.
- Workflow duplication possible for horizontal scaling.

Architecture mirrors microservices separation.

10. Security Considerations

Security features included:

- API key environment variable storage
- Webhook validation
- No hardcoded credentials
- Controlled access tokens
- Error logging without exposing sensitive data

This ensures safe automation practices.

11. Challenges Encountered

1. Handling PhantomBuster asynchronous job completion.
2. Preventing HubSpot rate limit violations.

3. Managing enrichment failures.
4. Avoiding duplicate CRM entries.
5. Maintaining workflow independence.

Solutions involved:

- Webhook event triggers
 - Batch processing logic
 - Upsert operations
 - Continue-on-fail configuration
 - Workflow decoupling
-

12. Key Engineering Principles Applied

- Separation of concerns
 - Event-driven architecture
 - Fault tolerance
 - API rate-limit compliance
 - Native CRM deduplication
 - Batch processing safety
 - Asynchronous execution design
-

13. Skills Demonstrated

- Workflow orchestration (n8n)
 - API integration
 - CRM automation
 - Event-driven systems
 - Asynchronous processing
 - Rate limit handling
 - Fault-tolerant design
 - Production automation architecture
 - Third-party SaaS integration
-

14. Real-World Applications

This architecture can be adapted for:

- Sales automation systems
 - Recruitment pipeline automation
 - Marketing campaign ingestion
 - Data warehousing ETL workflows
 - CRM synchronization engines
-

15. Future Improvements

- Queue-based system using Redis or Kafka
 - Multi-CRM support
 - Parallel enrichment pipelines
 - Retry policies with exponential backoff
 - Monitoring dashboard
 - SLA metrics tracking
 - Audit logging enhancements
-

This completes the detailed documentation for your Lead Ingestion Engine project.

Next project options:

- Knowledge Exchange SRD (Requirements Engineering System Design)
- RecklamRadar (Flutter + Firebase Cross-Platform App)

Say:

NEXT

And we continue.